

**THAPAR INSTITUTE OF ENGINEERING AND
TECHNOLOGY, PATIALA**

Department of Computer Science and Engineering (COPC)



Attendora

A Smart Attendance Management System

A Project Report Submitted in Partial Fulfilment of the Requirements
for the Course **Software Engineering**
5th Semester

Submitted By

Kshitiz Poudel Roll No. 1024170424

Biraj Chauhan Roll No. _____

Group: 3Q35

Under the Supervision of

Academic Session 2026–2027

Certificate

This is to certify that the project report entitled “**Attendora: A Smart Attendance Management System**” is submitted by **Kshitiz Poudel (1024170424)** and **Biraj Chauhan**, students of Group **3Q35**, 5th Semester, Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala, in partial fulfilment of the requirements for the course **Software Engineering**.

The work presented in this report is the original work of the students, carried out under my supervision, and has not been submitted elsewhere for the award of any other degree or diploma.

Date: _____

Signature of Supervisor

Name & Designation

Declaration

We hereby declare that the project work entitled “**Attendora: A Smart Attendance Management System**” is an authentic record of our own work carried out as part of the Software Engineering course during the 5th Semester, under the guidance of our supervisor. The material embodied in this report has not been submitted earlier, in full or in part, for the award of any degree or diploma at this or any other institution.

Kshitiz Poudel _____

Biraj Chauhan _____

Acknowledgment

We would like to express our sincere gratitude to our Software Engineering course faculty and the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, for their guidance, support, and valuable feedback throughout the development of this project.

We are also thankful to our peers in Group 3Q35 for their encouragement and constructive discussions, which helped us refine the design and implementation of Attendra. Finally, we acknowledge the open-source community behind Flutter and Firebase, whose tools and documentation made this project possible.

Kshitiz Poudel
Biraj Chauhan

Abstract

Manual attendance tracking in academic institutions is time-consuming, error-prone, and difficult to audit. **Attendora** is a cross-platform mobile and web application, built with **Flutter** and **Firebase**, that digitizes the entire attendance lifecycle for a classroom setting. The system supports four distinct roles — Student, Faculty, Admin, and Super Admin — each with a dedicated dashboard and permission scope enforced through Firebase Authentication and Firestore Security Rules.

Faculty members can create classes and start live attendance sessions; students check in to a session from their own device; attendance records are written in real time and aggregated into per-subject percentage statistics. When a student's attendance in a subject falls below the institutionally mandated **60% threshold**, the system flags them as eligible for detention, and a faculty member may formally detain the student from that course — a decision that is immediately reflected on the student's own dashboard. Administrators approve new faculty/student accounts and manage institutional data, while a Super Admin oversees multiple admin accounts across the platform.

This report documents the requirement analysis, system design, implementation, and testing of Attendora, and discusses the results, limitations, and scope for future enhancement of the system.

Contents

Certificate	1
Declaration	2
Acknowledgment	3
Abstract	4
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Report Organization	2
2 Literature Review	3
2.1 Traditional Attendance Methods	3
2.2 Existing Digital Solutions	3
2.3 Gaps Identified	4
3 Requirement Analysis	5
3.1 Functional Requirements	5
3.2 Non-Functional Requirements	6
3.3 Hardware and Software Requirements	6
3.4 Use Case Diagrams	6
4 System Design	8
4.1 System Architecture	8
4.2 Database Design	8
4.3 Sequence Diagram: Attendance Check-in	9
4.4 Sequence Diagram: Detention Flow	9
5 Implementation	11
5.1 Technology Stack	11

5.2	Project Structure	11
5.3	Module Description	12
5.3.1	Authentication Module	12
5.3.2	Class & Session Module	13
5.3.3	Attendance Module	13
5.3.4	Detention Module	13
5.3.5	Admin & Super Admin Modules	14
5.4	Security Rules	14
5.5	Branding and Theming	14
6	Testing	15
6.1	Testing Strategy	15
6.2	Test Cases	15
6.3	Results	16
7	Results and Discussion	17
7.1	User Interface Overview	17
7.2	Features Achieved	17
7.3	Limitations	17
8	Conclusion and Future Scope	20
8.1	Conclusion	20
8.2	Future Scope	20
	References	22
	Appendix A: Team Contribution	23

List of Figures

3.1	Use case diagram — Student and Faculty	7
3.2	Use case diagram — Admin and Super Admin	7
4.1	High-level system architecture of Attendora	8
4.2	Firestore collection schema and relationships	9
4.3	Sequence diagram for the attendance check-in flow	9
4.4	Sequence diagram for the detention workflow	10
7.1	UI mockup: Login screen	17
7.2	UI mockup: Student subject card showing the detained banner	18

List of Tables

3.1	Development environment requirements	6
5.1	Technology stack used in Attendora	11
6.1	Summary of functional test cases	16
7.1	Summary of implemented features against project objectives	19
1	Individual contribution summary	23

Chapter 1

Introduction

1.1 Background

Attendance tracking is one of the most routine yet administratively significant tasks in any academic institution. It directly affects student evaluation, eligibility for examinations, and institutional compliance reporting. Despite its importance, attendance in a large number of classrooms is still recorded manually — through paper registers or roll calls — a process that is slow, susceptible to proxy attendance, and difficult to aggregate into meaningful reports.

With the widespread availability of smartphones among students and faculty, there is a clear opportunity to replace manual registers with a lightweight, real-time digital system. **Attendora** was conceived as a Software Engineering project to explore this opportunity: a mobile-and-web attendance system that lets a faculty member start a session, lets students check in from their own device, and gives every stakeholder (student, faculty, admin) an accurate, real-time view of attendance data.

1.2 Problem Statement

Traditional attendance systems suffer from the following recurring problems:

- Manual roll-calls consume valuable classroom time.
- Paper registers are easy to manipulate (proxy attendance) and difficult to audit.
- Aggregating attendance percentages across a semester is a manual, error-prone spreadsheet exercise.
- There is no standard mechanism to automatically flag students who fall below a minimum attendance requirement and formally act on it (e.g., detention from a course).
- Institutions with multiple departments/admins have no unified oversight layer.

Attendora addresses each of these problems through a role-based, cloud-backed system with real-time synchronization.

1.3 Objectives

The primary objectives of this project are to:

1. Design and implement a role-based attendance system supporting Student, Faculty, Admin, and Super Admin roles.
2. Provide a session-based check-in mechanism that records attendance in real time.
3. Automatically compute per-subject attendance percentages for every student.
4. Enforce a minimum attendance threshold (60%) and allow faculty to formally detain students who fall below it.
5. Provide administrative workflows for account approval and institutional oversight.
6. Secure all data access using role-based Firestore security rules.

1.4 Scope of the Project

Attendora is developed as a cross-platform Flutter application, deployable to Android, iOS, and the web from a single codebase, backed by Firebase (Authentication, Cloud Firestore, and Storage). The current scope covers:

- Account registration and role-based login for Students and Faculty, subject to Admin approval.
- Class and session management by Faculty.
- Student check-in and attendance record generation.
- Attendance analytics and reporting for Faculty.
- Detention workflow for students below the 60% attendance threshold.
- Admin and Super Admin dashboards for institutional management.

Features such as biometric/geofenced attendance, push notifications, and multi-institution analytics are identified as future scope and discussed in Chapter 8.

1.5 Report Organization

The remainder of this report is organized as follows. Chapter 2 reviews existing attendance solutions and identifies the gaps Attendora addresses. Chapter 3 presents the requirement analysis. Chapter 4 describes the system architecture and design. Chapter 5 discusses the implementation details and key algorithms. Chapter 6 covers the testing strategy and results. Chapter 7 presents the results and a discussion of the working system. Chapter 8 concludes the report and outlines future scope.

Chapter 2

Literature Review

2.1 Traditional Attendance Methods

The most common attendance method in Indian academic institutions remains the manual paper register, where a faculty member calls out roll numbers or names and marks attendance by hand. While simple to implement, this method has well-documented drawbacks: it is time-consuming (often 5–10 minutes per lecture), prone to proxy attendance, and requires manual compilation at the end of a semester to compute per-student percentages.

2.2 Existing Digital Solutions

Several categories of digital attendance solutions exist in practice:

- **Biometric systems** (fingerprint/face recognition) — accurate and proxy-resistant, but require dedicated hardware at each classroom entry point, making them expensive to scale across a large institution.
- **RFID/Smart-card based systems** — reduce manual effort but require issuing and maintaining physical cards, and do not gracefully handle a lost or forgotten card.
- **Institution-specific ERP modules** — many universities have an attendance module bundled into a larger ERP system. These are often rigid, slow to update, and not designed with a mobile-first, real-time user experience in mind.
- **Generic mobile attendance apps** — consumer apps exist for general-purpose attendance tracking, but they are not tailored to the specific role hierarchy (Student/Faculty/Admin/Super Admin) or academic policies (such as a minimum attendance threshold with a formal detention consequence) that a college environment requires.

2.3 Gaps Identified

Based on the above review, the following gaps motivated the design of Attendora:

1. Lack of a low-cost, hardware-free solution that still resists trivial proxy attendance (via session-scoped, time-bound check-ins).
2. Absence of a built-in, automatic linkage between attendance percentage and institutional consequences (i.e., detention below 60%).
3. Poor real-time visibility for students into their own attendance standing.
4. No lightweight, cross-platform system that a small development team can deploy and iterate on quickly using modern cloud-backed tooling.

Attendora is designed specifically to close these gaps using a modern, cloud-native technology stack (Flutter + Firebase), discussed further in Chapters 4 and 5.

Chapter 3

Requirement Analysis

3.1 Functional Requirements

- FR1: The system shall allow a new user to register as either a Student or Faculty member.
- FR2: The system shall restrict registration to users with an institutional (@thapar.edu) or, for prototype purposes, a @gmail.com email address.
- FR3: The system shall require Admin approval before a newly registered Faculty or Student account becomes active.
- FR4: The system shall allow a Faculty member to create a class and start a time-bound attendance session for it.
- FR5: The system shall allow a Student to check in to an active session, recording a timestamped attendance entry.
- FR6: The system shall prevent duplicate check-ins for the same student within a single session.
- FR7: The system shall compute and display a running attendance percentage per subject for every student.
- FR8: The system shall allow a Faculty member to formally detain a student from a course once their attendance falls below 60%.
- FR9: The system shall display a clear “You have been detained from this course” notice to a detained student.
- FR10: The system shall allow an Admin to manage (approve/reject) Faculty and Student accounts within their institution.
- FR11: The system shall allow a Super Admin to oversee multiple Admin accounts and institutions.
- FR12: The system shall provide a permanent built-in administrator login for demonstration purposes.

3.2 Non-Functional Requirements

- **Usability:** The interface shall be simple enough for a first-time user to register and check in without external instructions.
- **Performance:** Attendance check-ins and dashboard updates shall reflect in real time (within a few seconds) using Firestore’s live listeners.
- **Security:** All data access shall be scoped by role using Firebase Authentication and Firestore Security Rules, so that a Student cannot read or modify another student’s records.
- **Portability:** The application shall run from a single Flutter codebase across Android, iOS, and the web.
- **Availability:** The backend (Firebase) shall provide high availability without the team needing to manage dedicated servers.
- **Maintainability:** The codebase shall follow a feature-first modular structure to keep each role’s functionality independently maintainable.

3.3 Hardware and Software Requirements

Table 3.1: Development environment requirements

Component	Specification
Framework	Flutter (Dart), Channel stable
Backend	Firebase (Authentication, Cloud Firestore, Storage)
State Management	Riverpod
IDE	Visual Studio Code / Android Studio
Version Control	Git & GitHub
Target Platforms	Android, iOS, Web (Chrome)
Minimum RAM (dev machine)	8 GB
Internet Connectivity	Required (cloud-backed data sync)

3.4 Use Case Diagrams

Given the number of actors involved, the use cases are presented as two focused diagrams for clarity: Figure 3.1 covers the Student and Faculty actors, and Figure 3.2 covers the Admin and Super Admin actors.

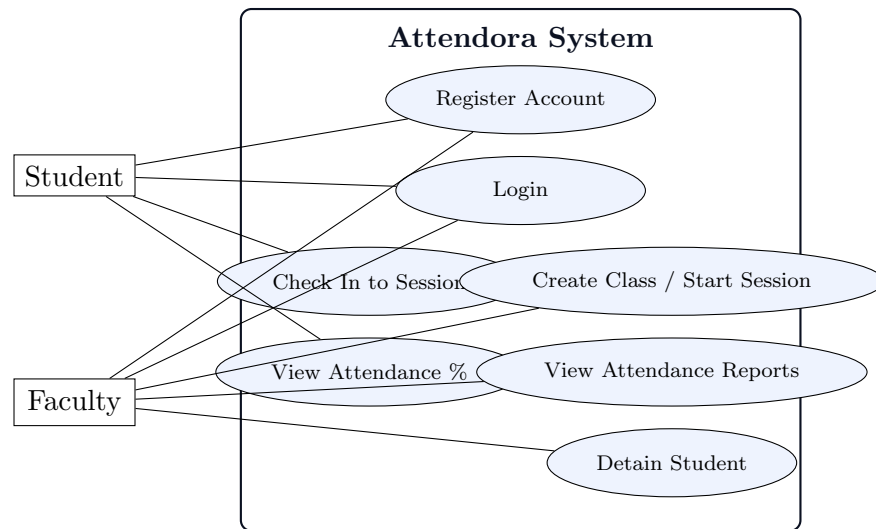


Figure 3.1: Use case diagram — Student and Faculty

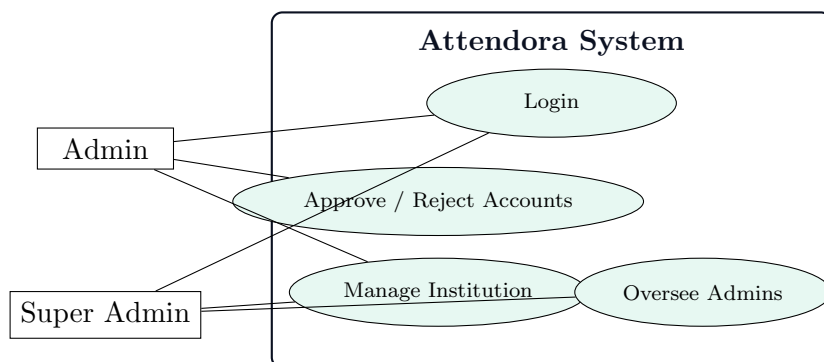


Figure 3.2: Use case diagram — Admin and Super Admin

Chapter 4

System Design

4.1 System Architecture

Attendora follows a client-heavy, serverless architecture. The Flutter client communicates directly with Firebase services — there is no custom backend server to maintain. Figure 4.1 illustrates this at a high level.

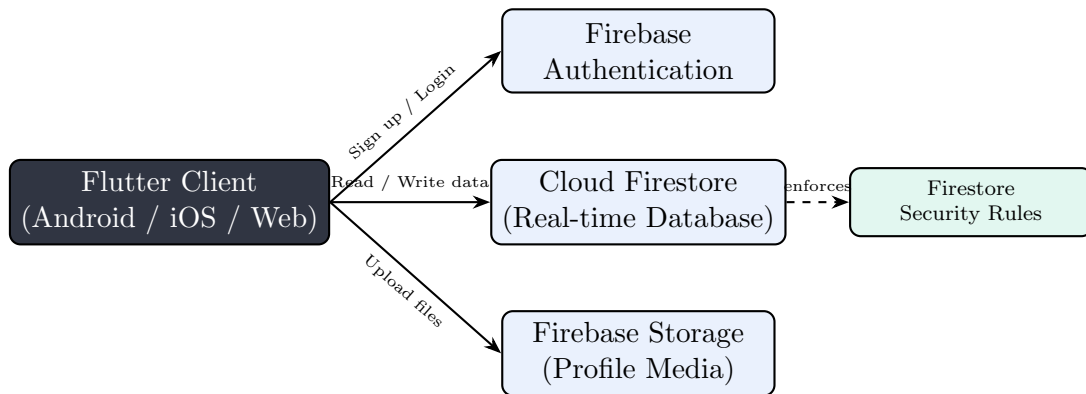


Figure 4.1: High-level system architecture of Attendora

The client is organized using a *feature-first* folder structure (`lib/features/<role>/...`), with a shared `lib/core` module for routing, services, and utilities. State is managed using **Riverpod**, which exposes reactive providers that automatically rebuild the UI whenever the underlying Firestore stream emits new data.

4.2 Database Design

Cloud Firestore is a document-oriented NoSQL database. Attendora’s data model is organized into five primary collections, shown in Figure 4.2.

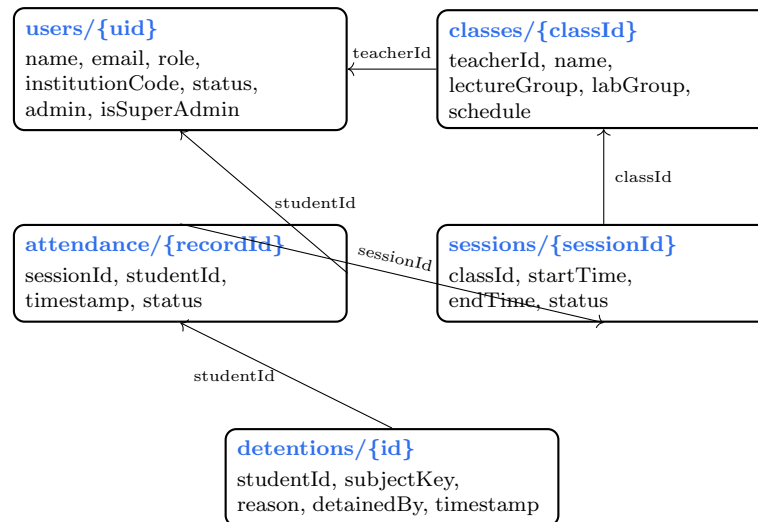


Figure 4.2: Firestore collection schema and relationships

4.3 Sequence Diagram: Attendance Check-in

Figure 4.3 illustrates the flow when a Faculty member starts a session and a Student checks in.

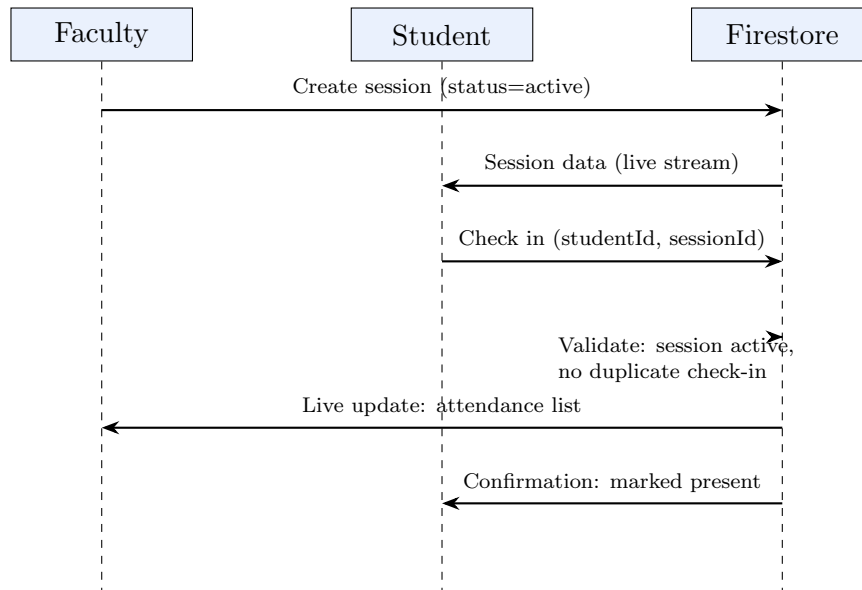


Figure 4.3: Sequence diagram for the attendance check-in flow

4.4 Sequence Diagram: Detention Flow

Figure 4.4 shows how a student is detained once their attendance drops below the 60% threshold.

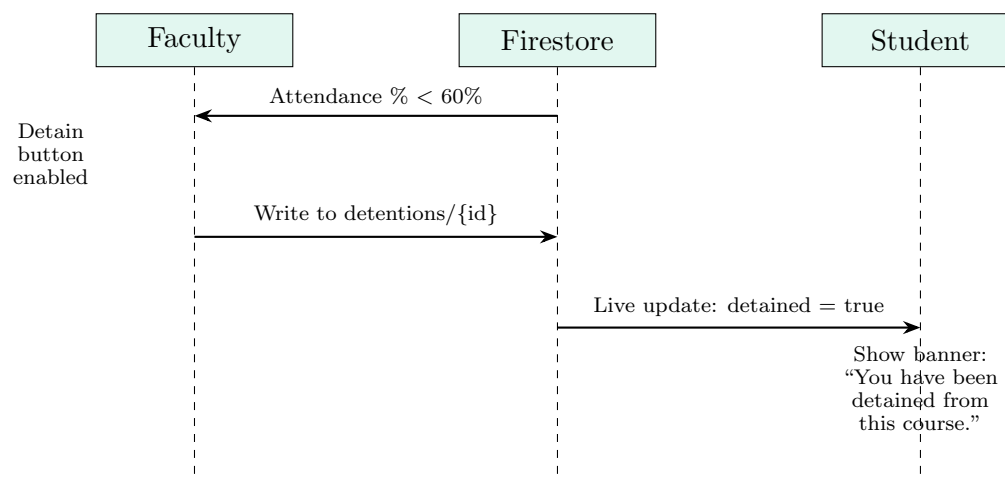


Figure 4.4: Sequence diagram for the detention workflow

Chapter 5

Implementation

5.1 Technology Stack

Attendora is implemented using the following technologies:

Table 5.1: Technology stack used in Attendora

Layer	Technology
UI Framework	Flutter (Dart)
State Management	Riverpod (<code>ConsumerWidget</code> , <code>StreamProvider</code>)
Authentication	Firebase Authentication (email/password, Google Sign-In)
Database	Cloud Firestore (NoSQL, real-time listeners)
File Storage	Firebase Storage
Typography	Google Fonts (Outfit)
Testing	<code>flutter_test</code>
Version Control	Git & GitHub

5.2 Project Structure

The codebase follows a feature-first architecture:

Listing 5.1: Simplified project directory structure

```
1 lib/
2   core/
3     services/           Shared Firebase/service helpers
4     utils/
5     router.dart         Role-based route guarding
6   features/
7     auth/               Login, signup, role-based
8     redirection
9     student/           Student dashboard, subjects, check-
10    in
```

9	teacher/	Faculty dashboard, sessions, reports, detention
10	dashboard/	Admin dashboard and management pages
11	super_admin/	Super Admin oversight pages
12	splash/	Splash screen and branding
13	shared/	Reusable widgets (cards, dialogs)

5.3 Module Description

5.3.1 Authentication Module

Handles registration, login, and role-based routing. New users choose either *Student* or *Faculty* at sign-up; both roles require Admin approval before becoming active. For the prototype, both @thapar.edu and @gmail.com email addresses are accepted:

Listing 5.2: Email domain validation (repository.dart)

```

1 bool _isThaparEmail(String email) {
2   final lower = email.toLowerCase();
3   // Prototype: allow both the institutional domain and
4     gmail.com
5   return lower.endsWith('@thapar.edu') || lower.endsWith('@gmail.com');
6 }

```

For demonstration purposes, a permanent built-in administrator account is also supported, bypassing Firebase entirely so that grading/demo access is never dependent on the live database's state:

Listing 5.3: Permanent admin login bypass (providers.dart)

```

1 if (rollNumber.trim() == '1234567890' && password == '
2   ADMIN123') {
3   state = state.copyWith(
4     loading: false,
5     role: UserRole.admin,
6     uid: 'builtin-admin',
7     email: 'admin@attendora.local',
8     displayName: 'Administrator',
9     rollNumber: rollNumber.trim(),
10    isAdmin: true,

```

```

10     isSuperAdmin: false,
11     institutionCode: null,
12     error: null,
13 );
14 return;
15 }

```

5.3.2 Class & Session Module

Allows a Faculty member to create a class (tagged with a lecture group, e.g., 3Q31--35, and a lab group, e.g., 3Q31-3Q35) and start a time-bound attendance session against it.

5.3.3 Attendance Module

Records a timestamped check-in for each student against an active session, with duplicate check-in prevention. Attendance percentage per subject is derived as:

$$\text{Attendance \%} = \frac{\text{Sessions Attended}}{\text{Total Sessions Held}} \times 100 \quad (5.1)$$

5.3.4 Detention Module

Once a student's attendance in a subject falls below 60%, the Faculty dashboard flags them as eligible for detention:

Listing 5.4: Detention eligibility check (`teacher_students_page.dart`)

```

1 final eligible = percentage < 60.0;
2 ...
3 if (percentage >= 60) return Colors.orange;

```

If the Faculty member confirms the detention, a record is written to the `detentions` collection. The student's own dashboard listens for this flag per subject and displays a banner accordingly:

Listing 5.5: Detained flag attached to a student's subject data (`student/providers.dart`)

```

1 return {
2   ...subject,
3   'stats': {
4     'attended': attendedCount,

```

```
5      'total': totalSessions,
6      'percentage': totalSessions > 0
7          ? (attendedCount / totalSessions * 100)
8          : 0.0,
9  },
10  'detained': detainedKeys.contains(
11      '${subjectName.trim().toLowerCase()}|${subjectGroup.
12          trim().toLowerCase()}',
13  ),
14  };
```

On the student’s subjects screen, this flag renders a persistent warning banner: *“You have been detained from this course.”*

5.3.5 Admin & Super Admin Modules

The Admin dashboard allows approval/rejection of pending Faculty and Student accounts and management of institution-scoped data. The Super Admin dashboard provides a higher-level oversight view across multiple Admin accounts and institutions, with its own Firestore-rule-enforced access boundary.

5.4 Security Rules

Firestore Security Rules enforce that a user can only read/write data appropriate to their role — for example, a Student cannot write to another student’s attendance record, and only a Super Admin’s user document (verified by a rule matching their email) can access cross-institution data.

5.5 Branding and Theming

The application’s visual identity is built around a dedicated Attendora logo (a QR-scan motif combined with a checkmark and two figures, symbolizing the “Scan · Mark · Attend” workflow). The colour palette — a primary blue (#2F6FED) and a teal-green accent (#10B981) — is applied consistently across the splash screen, dashboards, and interactive elements to give the application a cohesive, professional appearance.

Chapter 6

Testing

6.1 Testing Strategy

Attendora was tested using a combination of:

- **Unit tests** on core logic — authentication state transitions, attendance percentage calculation, and detention eligibility — using `flutter_test`.
- **Manual functional testing** across all four roles (Student, Faculty, Admin, Super Admin) on both the Chrome (web) and Android targets.
- **End-to-end scenario testing** covering the complete workflow: registration → approval → session creation → check-in → reporting → detention.

6.2 Test Cases

Test ID	Test Description	Expected Result	Status
TC-01	Register with a valid @thapar.edu email	Account created with status “pending”	Pass
TC-02	Register with a valid @gmail.com email (prototype)	Account created with status “pending”	Pass
TC-03	Register with an unsupported email domain	Registration rejected with a clear error	Pass
TC-04	Login with unapproved account	Access denied / pending message shown	Pass
TC-05	Admin approves a pending Faculty account	Faculty can now log in and access dashboard	Pass
TC-06	Faculty creates a class and starts a session	Session appears as “active” in Firestore	Pass

Test ID	Test Description	Expected Result	Status
TC-07	Student checks in to an active session	Attendance record created; visible on Faculty's live view	Pass
TC-08	Student attempts a duplicate check-in	Duplicate check-in blocked	Pass
TC-09	Student checks in to a closed session	Check-in rejected with an appropriate message	Pass
TC-10	Attendance percentage calculation for a student with 3/5 sessions attended	60.0% displayed	Pass
TC-11	Faculty views a student with attendance below 60%	"Detain" action becomes available	Pass
TC-12	Faculty detains a student below 60%	Record written to detentions ; reflected on student dashboard	Pass
TC-13	Detained student opens the subject screen	"You have been detained from this course." banner shown	Pass
TC-14	Login using the permanent admin credentials (ID: 1234567890)	Immediate Admin dashboard access, independent of database state	Pass
TC-15	Student attempts to access Admin-only routes	Access blocked by role-based router guard	Pass

Table 6.1: Summary of functional test cases

6.3 Results

All test cases listed in Table 6.1 passed during manual verification. No blocking defects were observed in the core attendance-marking and detention workflows at the time of writing. Minor UI polish items (loading-state consistency, empty-state messaging) were addressed iteratively during development.

Chapter 7

Results and Discussion

7.1 User Interface Overview

Figures 7.1 and 7.2 present representative mockups of two key screens in the working application, reflecting Attendora’s final colour scheme (primary blue #2F6FED and teal-green accent #10B981).

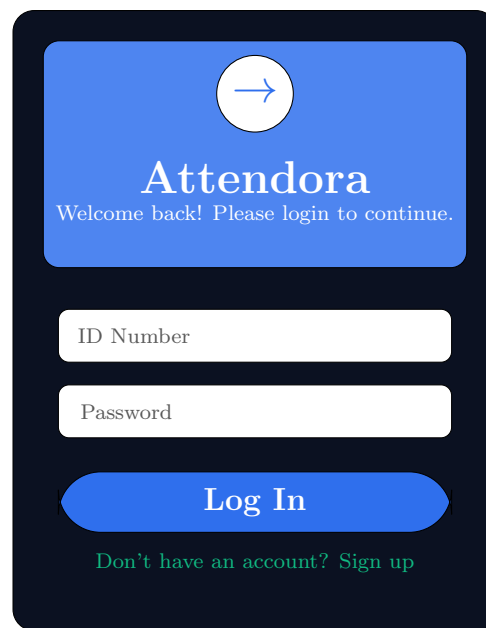


Figure 7.1: UI mockup: Login screen

7.2 Features Achieved

7.3 Limitations

- The current check-in mechanism relies on manual selection of an active session rather than QR-code scanning or geofencing, leaving room for a student to check in on behalf of a nearby peer.



Figure 7.2: UI mockup: Student subject card showing the detained banner

- The dual-domain (`gmail.com`) login is intended strictly as a prototype convenience and would need to be removed or restricted before a production rollout.
- The permanent admin credentials, while useful for demonstration, represent a hardcoded backdoor that must not be shipped in a production build.
- Attendance analytics are currently limited to percentage and history; trend visualizations (e.g., week-over-week charts) are not yet implemented.

Table 7.1: Summary of implemented features against project objectives

Feature	Status
Role-based registration (Student / Faculty) with Admin approval	Complete
Dual-domain login (@thapar.edu / @gmail.com) for prototype	Complete
Class and session creation by Faculty	Complete
Real-time student check-in and attendance recording	Complete
Per-subject attendance percentage calculation	Complete
Automatic detention eligibility flag below 60%	Complete
Faculty-initiated detention with student-side notification	Complete
Admin approval and institution management	Complete
Super Admin oversight dashboard	Complete
Permanent built-in admin login for demonstration	Complete
Consistent Attendora branding (logo, colour palette)	Complete

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

Attendora successfully demonstrates a modern, role-based approach to classroom attendance management. By replacing manual paper registers with a real-time, cloud-backed Flutter and Firebase application, the system reduces the time and administrative overhead of tracking attendance while introducing a direct, automated link between attendance percentage and institutional policy — specifically, the 60% minimum attendance requirement and the associated detention workflow. The four-role architecture (Student, Faculty, Admin, Super Admin) mirrors the real hierarchy of a college department, and Firestore’s real-time syncing ensures every stakeholder sees an up-to-date view of attendance data without manual refreshes or batch reports.

This project also served as a practical exercise in applying Software Engineering principles — requirement analysis, modular system design, iterative implementation, and structured testing — to a real-world problem with a working, demonstrable end product.

8.2 Future Scope

The following enhancements are identified as valuable next steps beyond the current prototype:

- **QR-code or geofenced check-in** to further reduce the possibility of proxy attendance.
- **Push notifications** to alert students proactively as their attendance approaches the 60% threshold, before detention becomes necessary.
- **Advanced analytics** — trend charts, semester-over-semester comparisons, and exportable reports for institutional record-keeping.
- **Removal of prototype-only conveniences** (the `gmail.com` login domain and the hardcoded admin credentials) before any production deployment.

- **Multi-institution support** with a proper Super Admin-managed onboarding flow for new institutions.
- **Automated CI/CD** with a test suite run on every commit, to catch regressions earlier as the codebase grows.

References

- [1] Flutter Documentation, Google. <https://docs.flutter.dev>
- [2] Firebase Documentation, Google. <https://firebase.google.com/docs>
- [3] Cloud Firestore Data Model, Google. <https://firebase.google.com/docs/firestore/data-model>
- [4] Riverpod Documentation. <https://riverpod.dev>
- [5] Firebase Authentication Guides, Google. <https://firebase.google.com/docs/auth>
- [6] Firestore Security Rules, Google. <https://firebase.google.com/docs/firestore/security/get-started>
- [7] Sommerville, I. *Software Engineering*, 10th Edition, Pearson Education.
- [8] Pressman, R. S. *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill.

Appendix A: Team Contribution

Table 1: Individual contribution summary

Member	Primary Contribution
Kshitiz Poudel	Application architecture, Flutter/Firebase implementation, authentication module, attendance and detention logic, branding, and report preparation.
Biraj Chauhan	Requirement analysis, testing, admin/super-admin module support, and documentation review.

Note: This summary is provided for academic record-keeping and can be adjusted to reflect the actual division of work between team members.